



Vite & Gourmand

Application traiteur — Bordeaux

Évan OROFINO

TP — Développeur Web et Web Mobile

Studi · Promotion 2026

ECF 2 — Hiver 2025 / Printemps 2026

16 mai 2026

Liens des livrables

Application déployée :

Déploiement sur Render en cours — voir
docs/deploiement/GUIDE_DEPLOIEMENT_RENDER.md

Code source GitHub PUBLIC :

<https://github.com/EvanOROFINO/vitegourmand>

Identifiants de démonstration (mot de passe : Password123!)

Administrateur : `admin@vitegourmand.fr` · Employé : `employe@vitegourmand.fr` · Client :
`user@vitegourmand.fr` · Client 2 : `jean@example.fr`

Sommaire

1. Manuel d'utilisation
2. Charte graphique
3. Maquettes haute fidélité
4. Documentation technique
5. Modèle Conceptuel de Données
6. Diagramme cas d'utilisation
7. Gestion de projet
8. Kanban

1

Manuel d'utilisation

Manuel d'utilisation — Vite & Gourmand

Application web de prise de commande pour le traiteur Vite & Gourmand (Bordeaux).

1. Accès à l'application

- **URL en ligne** : (à compléter après déploiement)
- **URL en local** : `http://localhost:8000`

2. Comptes de démonstration

Tous les comptes ont pour mot de passe : **Password123!**

Rôle	Email	Capacités
 Administrateur	<code>admin@vitegourmand.fr</code>	Toutes les actions de l'application
 Employé	<code>employe@vitegourmand.fr</code>	Gestion menus / commandes / avis / horaires
 Utilisateur	<code>user@vitegourmand.fr</code>	Consulter, commander, donner un avis
 Utilisateur 2	<code>jean@example.fr</code>	Idem ci-dessus

3. Parcours visiteur (sans compte)

3.1 Découvrir l'entreprise

Sur la page d'accueil, le visiteur découvre :

- Une présentation de Julie & José
- Trois atouts (25 ans d'expérience, produits locaux, sur-mesure)
- Les avis clients **validés** par un employé

3.2 Consulter les menus

Cliquer sur « **Nos menus** » dans la navbar.

- 6 menus sont proposés par défaut (Tradition Bordelaise, Noël Festif, Végétarien Gourmand, Pâques Familial, Été Frais, Évènement Prestige).
- Des **filtres dynamiques** sont disponibles (sans rechargement de page) :
 - Prix minimum / maximum (€/personne)
 - Thème (Classique, Noël, Pâques, Évènement, Été)

- Régime (Classique, Végétarien, Végan, Sans gluten, Halal)
- Nombre de personnes minimum

3.3 Voir le détail d'un menu

En cliquant sur « **Voir le détail** » :

- Galerie d'images
- Description complète
- Composition (entrée / plat / dessert avec allergènes)
- Liste agrégée des allergènes du menu
- **Conditions du menu** (mises en avant en bandeau jaune)
- Prix par personne, nombre minimum, stock restant
- Bouton « **Commander** » (redirige vers connexion si visiteur non authentifié)

3.4 Contacter l'entreprise

Cliquer sur « **Contact** » dans la navbar :

- Formulaire avec titre, e-mail, message
- Une fois envoyé : confirmation à l'écran + e-mail à l'équipe Vite & Gourmand

3.5 Mentions légales / CGV

Accessible depuis le footer.

4. Parcours utilisateur (compte client)

4.1 Créer un compte

Cliquer sur « **Connexion** » puis « **Créez-en un** ».

Champs requis :

- Prénom, nom
- E-mail (servira d'identifiant)
- Téléphone (GSM)
- Adresse postale, ville, pays
- Mot de passe sécurisé : **10 caractères minimum, 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial**

→ Un mail de bienvenue automatique est envoyé.

4.2 Se connecter

Email + mot de passe. En cas d'oubli, lien « **Mot de passe oublié** » qui envoie un mail avec un lien de réinitialisation valable 1 heure.

4.3 Passer une commande

1. Sur la page détail d'un menu, cliquer sur « **Commander** »
2. Le formulaire est pré-rempli avec les informations du compte
3. Renseigner :
 - Date et heure de la prestation (J+1 minimum)
 - Adresse de livraison + ville
 - Distance depuis Bordeaux en km (si livraison hors Bordeaux)
 - Nombre de personnes (minimum imposé par le menu)
4. Le **récapitulatif tarifaire** se met à jour en temps réel :
 - Prix menu = `nombre × prix_par_personne`
 - **Réduction -10 %** automatique si `nombre_personnes ≥ minimum + 5`
 - **Frais livraison = 5 € + 0,59 €/km** si livraison hors Bordeaux
5. Valider → numéro de commande unique généré + mail de confirmation

4.4 Suivre ses commandes

Espace « **Mon espace** » → « **Mes commandes** ».

Pour chaque commande, accès au détail :

- Informations
- Tarification
- **Suivi temps réel** des changements de statut (timeline)

Statuts possibles

```
en_attente → accepte → en_preparation → en_cours_de_livraison → livre → terminee
                ↓
                (avec prêt matériel)
                ↓
en_attente_retour_materiel → terminee
```

4.5 Modifier ou annuler une commande

Tant que le statut est `en_attente` (non traitée par un employé) :

- Modification possible (sauf le menu choisi)
- Annulation possible

Une fois le statut `accepte` **ou plus** : il faut contacter l'équipe.

4.6 Donner un avis

Une fois la commande `terminee` , un mail invite à laisser un avis.

Depuis le détail de la commande, cliquer sur « **Donner mon avis** » :

- Note de 1 à 5 étoiles

- Commentaire libre

L'avis est créé en `en_attente` de modération. Une fois validé par un employé, il s'affiche sur la page d'accueil.

5. Parcours employé

5.1 Tableau de bord

URL : `/employe` . Vue principale = **liste des commandes** avec :

- Filtre par statut
- Recherche client (nom, prénom, e-mail, n° commande)
- Action « **Changer statut** » par dropdown

5.2 Gérer les commandes

L'employé peut faire évoluer une commande dans n'importe quel statut. Mails automatiques envoyés à certains paliers :

- `en_attente_retour_materiel` (si prêt matériel) → mail avec mention 600 € si non rendu sous 10 jours ouvrés
- `terminee` → mail invitant à donner un avis

Annulation : possible mais nécessite un **motif** + **mode de contact** (GSM ou mail).

5.3 Gérer les menus

URL : `/employe/menus` .

- Création / édition / désactivation
- Choix du thème, régime, nombre de personnes minimum, prix, conditions, stock
- Sélection des plats inclus (case à cocher pour chaque plat)

5.4 Gérer les plats

URL : `/employe/plats` .

- Ajout d'un plat (titre + type entrée/plat/dessert)
- Suppression

5.5 Gérer les horaires

URL : `/employe/horaires` . Modification des heures d'ouverture/fermeture pour chaque jour de la semaine.

5.6 Modérer les avis

URL : `/employe/avis` . Liste des avis en attente de validation.

Pour chaque avis : aperçu (note + commentaire) + boutons **Valider** / **Refuser**.

6. Parcours administrateur

L'admin a tous les droits d'un employé **plus** :

6.1 Gérer les comptes employés

URL : `/admin/employes` .

Création d'un employé : prénom, nom, e-mail, mot de passe initial.

- Le mot de passe **n'est PAS envoyé par e-mail** (à transmettre en main propre, conformément à l'énoncé)
- Un mail de bienvenue avertit l'employé de l'existence de son compte

Désactivation / réactivation d'un compte employé.

⚠ Conformément à l'énoncé : **aucun compte administrateur ne peut être créé depuis l'application**. L'admin initial est créé manuellement en BDD via le seed SQL.

6.2 Statistiques (BDD NoSQL)

URL : `/admin/statistiques` .

Graphique en barres (Chart.js) montrant le **nombre de commandes par menu** et le **chiffre d'affaires** correspondant. Ces données viennent de la base **MongoDB** (collection `commandes_par_menu`).

6.3 Chiffre d'affaires

URL : `/admin/chiffre-affaires` .

Filtres :

- Période (date début / date fin)
- Menu spécifique

KPIs : nombre total de commandes + CA total sur la période. Détail par menu en tableau.

7. Sécurité et conformité

Mesure	Implémentation
Hachage mot de passe	bcrypt (PHP <code>password_hash</code> , coût 10)
Protection CSRF	Jetons uniques par session, vérifiés sur chaque POST
Protection XSS	Échappement HTML systématique via <code>e()</code> (<code>htmlspecialchars</code>)
Protection SQL injection	Requêtes préparées PDO avec paramètres nommés
Sessions	<code>cookie_httponly</code> , <code>cookie_samesite=Lax</code> , <code>use_strict_mode</code> , regenerate sur login
RBAC	Rôles <code>utilisateur</code> , <code>employe</code> , <code>administrateur</code> vérifiés par <code>Auth::requireRole()</code>
Headers HTTP	<code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>Referrer-Policy</code> , <code>Content-Security-Policy</code>
RGPD	Mentions légales + droit d'accès/rectification/suppression mentionné
RGAA	Skip-link, aria-labels, contrastes WCAG AA, navigation clavier

8. Dépannage

MySQL ne démarre pas

```
Start-Process "C:\xampp\mysql\bin\mysqld.exe" -ArgumentList "--defaults-file=C:\xampp\mysql\bin\m
```

MongoDB ne démarre pas

```
& "C:\Program Files\MongoDB\Server\8.2\bin\mongod.exe" --dbpath "C:\data\db" --port 27017
```

Réinitialiser la BDD

```
Get-Content sql/01_schema.sql | & "C:\xampp\mysql\bin\mysql.exe" -u root  
Get-Content sql/02_seed.sql | & "C:\xampp\mysql\bin\mysql.exe" -u root
```

Voir les mails envoyés en mode dev

Les mails sont écrits dans `var/mail-debug/` (un fichier HTML par mail).

2

Charte graphique

Charte graphique — Vite & Gourmand

1. Identité visuelle

L'identité visuelle de **Vite & Gourmand** s'appuie sur une atmosphère **chaleureuse, conviviale et raffinée**, en cohérence avec l'image d'un traiteur familial bordelais en activité depuis 25 ans.

Les couleurs choisies évoquent :

- la **terre cuite** des plats traditionnels du Sud-Ouest,
- le **vert sauge** des aromates et de la nature,
- le **doré** des bonnes tablées,
- le **brun chocolat** et la **crème** des intérieurs cosy.

2. Palette de couleurs

Rôle	Nom	Hexadécimal	Aperçu
Primaire	Terracotta	#C8552B	Boutons CTA, liens, accent principal
Primaire foncé	Terracotta dark	#A03F1E	Hover des boutons primaires
Secondaire	Vert sauge	#6E8B5C	Badges secondaires, charts admin
Secondaire foncé	Sauge dark	#556C47	Hover des liens sauge
Accent	Or doux	#E5C26F	Étoiles d'avis, titres footer
Fond clair	Crème	#FAF6F0	Fond du body, tables alternées
Fond foncé	Brun chocolat	#2A2622	Footer, hero overlay
Texte principal	Brun foncé	#2A2622	Tous les textes courants
Texte secondaire	Brun moyen	#6B5E54	Légendes, helpers
Bordures	Beige clair	#E2D6C7	Cadres, séparateurs
Succès	Vert	#4A8B4A	Alertes succès
Erreur	Rouge brique	#C0392B	Alertes erreur, champs requis
Avertissement	Or chaud	#D9A441	Alertes warning, allergènes
Info	Bleu doux	#4A7AA9	Alertes info

Contraste WCAG AA : tous les couples texte/fond utilisés respectent un ratio $\geq 4,5:1$ (vérifié pour #2A2622 sur #FAF6F0 et pour le primaire #C8552B sur fond crème).

3. Typographie

Usage	Famille	Poids	Source
Titres (h1, h2, h3, h4)	Playfair Display	600, 700	Google Fonts
Corps de texte	Inter	400, 500, 600, 700	Google Fonts
Code / monospace	system-ui monospace	400	Native

Règles d'échelle (mobile-first, fluide) :

- `h1` : `clamp(2rem, 4vw, 3rem)`
- `h2` : `clamp(1.5rem, 3vw, 2.2rem)`
- `h3` : `1.4rem`
- Corps : `1rem` (16 px) avec `line-height: 1.6`

4. Iconographie

Bibliothèque officielle : **Bootstrap Icons 1.11** (chargée via CDN jsdelivr).

Icônes principales utilisées :

- `bi-cup-hot-fill` — logo et navbar
- `bi-people-fill` — convives
- `bi-cart-plus` — bouton commander
- `bi-star-fill` , `bi-star` — système de notation
- `bi-funnel-fill` — filtres
- `bi-clock-fill` — horaires
- `bi-bag-fill` — commandes
- `bi-currency-euro` — chiffre d'affaires
- `bi-bar-chart-fill` — statistiques

5. Composants d'interface

Boutons

- **Primaire** : fond `#C8552B` , texte blanc, `border-radius: 6px` , `padding: 0.7rem 1.5rem`
- **Secondaire** : fond blanc, texte foncé, bordure `#E2D6C7`
- **Danger** : fond `#C0392B` , texte blanc

Cartes

- Fond blanc
- `border-radius: 10px`

- `box-shadow: 0 1px 3px rgba(0,0,0,0.08)` au repos
- `box-shadow: 0 4px 12px rgba(0,0,0,0.10)` au hover

Formulaires

- Champs : padding `0.7rem`, bordure `2px solid #E2D6C7`
- Focus : bordure remplacée par `#C8552B`
- Champs requis : marquage `*` rouge à côté du label

Alerts

Bordure gauche colorée (`4px`) selon le type :

- Succès → vert
- Erreur → rouge brique
- Info → bleu
- Warning → or

6. Accessibilité (RGAA)

Critère	Implémentation
Skip link	<code></code> en haut de chaque page
Aria-labels	Sur tous les boutons icône et formulaires complexes
Aria-expanded	Sur le bouton menu mobile
Aria-live	Sur les alerts (rôle <code>alert</code>)
Focus visible	Outline <code>3px</code> <code>#E5C26F</code> sur tous les éléments interactifs
Contrastes	Vérifiés $\geq 4,5:1$
Hiérarchie	Un seul <code><h1></code> par page, hiérarchie respectée
Navigation clavier	Toutes les actions accessibles à la touche <code>Tab</code>
<code>prefers-reduced-motion</code>	Animations limitées aux transitions essentielles

7. Maquettes

Maquettes desktop (3)

1. **Page d'accueil** — Hero avec photo de plats, présentation de l'équipe, avis clients
2. **Liste des menus** — Filtres dynamiques + grille de cartes menus
3. **Détail d'un menu** — Galerie + composition + bouton commande

Maquettes mobile (3)

- 1. **Page d'accueil mobile** — Stack vertical, navbar repliable
- 2. **Liste des menus mobile** — Filtres au-dessus, cartes en pile
- 3. **Tunnel de commande mobile** — Formulaire en étapes avec récap dynamique

Les maquettes finales sont exportées au format PNG dans le dossier `docs/charte/maquettes/`.

8. Responsive design

Breakpoints utilisés :

Écran	Largeur	Approche
Mobile	< 600 px	Stack vertical, menu repliable
Tablette	600 px - 1024 px	Grilles 2 colonnes, navbar pleine
Desktop	> 1024 px	Grilles 3 colonnes, hero pleine largeur

Container : `max-width: 1200px` (large) / `720px` (étroit).

3

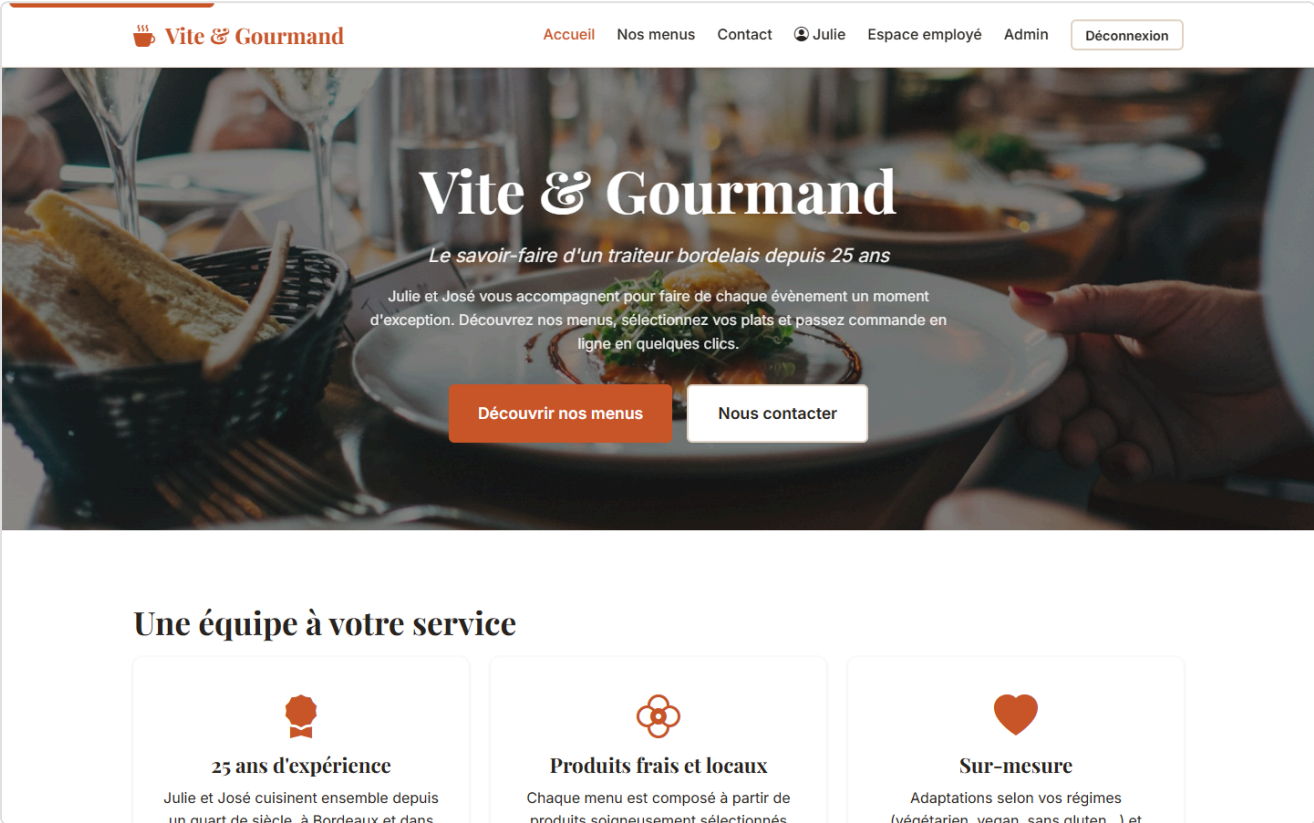
Maquettes haute fidélité

Maquettes — Vite & Gourmand

Maquettes **haute fidélité** générées via Puppeteer (script `tools/screenshots.cjs`). Toutes les pages sont capturées en **desktop (1440×900)** et **mobile (414×896)**.

1. Page d'accueil

Desktop



Vite & Gourmand

*Le savoir-faire d'un traiteur bordelais
depuis 25 ans*

Julie et José vous accompagnent pour faire de
chaque évènement un moment d'exception.
Découvrez nos menus, sélectionnez vos plats et
passez commande en ligne en quelques clics.

Découvrir nos menus

Nous contacter

Une équipe à votre service



2. Liste des menus avec filtres

Desktop

 **Vite & Gourmand**

[Accueil](#) [Nos menus](#) [Contact](#) [Julie](#) [Espace employé](#) [Admin](#) [Déconnexion](#)

Nos menus

Découvrez nos menus traiteurs, des plus classiques aux plus festifs.

▼ Filtrer

Prix min. (€/pers)

Prix max. (€/pers)

Thème

Régime

Convives

Tous

Tous

ex : 10

Réinitialiser



No???

Classique

Menu 22x22 Essie



??v??nement

Classique

Menu 22x22nement Drectice



No???

Classique

Menu de No22? Festif

Mobile

Nos menus

Découvrez nos menus traiteurs, des plus classiques aux plus festifs.

Filtrer

Prix min. (€/pers)

Prix max. (€/pers)

Thème

Tous



Régime

Tous



Convives

3. Détail d'un menu

Desktop

404

La page que vous cherchez n'existe pas.

[Retour à l'accueil](#)

404

La page que vous cherchez n'existe pas.

[Retour à l'accueil](#)

4. Page de connexion

Desktop

 **Vite & Gourmand**

[Accueil](#) [Nos menus](#) [Contact](#) [Julie](#) [Espace employé](#) [Admin](#) [Déconnexion](#)

Bonjour Julie 🙌

Voici votre espace personnel.



Mes commandes
0 commande



Mon profil
Modifier mes informations



Découvrir les menus
Passer une nouvelle commande

Erreur : Class "App\Models\Horaire" not found

```
#0 C:\Users\evano\Desktop\vitegourmand\views\layouts\main.php(36): require()  
#1 C:\Users\evano\Desktop\vitegourmand\src\Core\Controller.php(28): require('C:\\Users\\evano\\...')  
#2 C:\Users\evano\Desktop\vitegourmand\src\Controllers\UserController.php(19): App\Core\Controller->view('user/dashboard', Array)  
#3 C:\Users\evano\Desktop\vitegourmand\src\Core Router.php(60): App\Controllers\UserController->dashboard()  
#4 C:\Users\evano\Desktop\vitegourmand\src\Core Router.php(47): App\Core Router->call(Array, Array)  
#5 C:\Users\evano\Desktop\vitegourmand\public\index.php(32): App\Core Router->dispatch('GET', '/mon-espace')  
#6 {main}
```


Bonjour Julie 🖐️

Voici votre espace personnel.



Mes commandes

0 commande



Mon profil

Modifier mes informations



Découvrir les menus

Passer une nouvelle commande

5. Création de compte

Desktop



[Accueil](#) [Nos menus](#) [Contact](#) [Julie](#) [Espace employé](#) [Admin](#) [Déconnexion](#)

Bonjour Julie 🙌

Voici votre espace personnel.



Mes commandes

0 commande



Mon profil

Modifier mes informations



Découvrir les menus

Passer une nouvelle commande

Erreur : Class "App\Models\Horaire" not found

```
#0 C:\Users\evano\Desktop\vitegourmand\views\layouts\main.php(36): require()  
#1 C:\Users\evano\Desktop\vitegourmand\src\Core\Controller.php(28): require('C:\\Users\\evano\\...')  
#2 C:\Users\evano\Desktop\vitegourmand\src\Controllers\UserController.php(19): App\Core\Controller->view('user/dashboard', Array)  
#3 C:\Users\evano\Desktop\vitegourmand\src\Core Router.php(60): App\Controllers\UserController->dashboard()  
#4 C:\Users\evano\Desktop\vitegourmand\src\Core Router.php(47): App\Core Router->call(Array, Array)  
#5 C:\Users\evano\Desktop\vitegourmand\public\index.php(32): App\Core Router->dispatch('GET', '/mon-espace')  
#6 {main}
```


Bonjour Julie 🖐️

Voici votre espace personnel.



Mes commandes

0 commande



Mon profil

Modifier mes informations



Découvrir les menus

Passer une nouvelle commande

6. Espace employé

Desktop



[Accueil](#) [Nos menus](#) [Contact](#) [Julie](#) [Espace employé](#) [Admin](#)

Déconnexion

Espace employé — Commandes



Menus



Plats



Horaires



Avis (0 en attente)

Statut

Tous

Rechercher (client, n° commande)

Filtrer

N°	Client	Menu	Date	Total	Statut	Actions
CMD-20260507-E58E96	Marie Lambert user@vitegourmand.fr	Menu Tradition Bordelaise (x12)	22 mai 2026	486,00 €	Terminée	→ Changer statut OK
CMD-20260420-	Jean Bernard		10 mai	325.90	En	→ Changer statut

Espace employé — Commandes



Menus



Plats



Horaires



Avis (0 en attente)

Reproduire ces captures

```
# Démarrer le serveur PHP local
php -S localhost:8001 -t public

# Dans un autre terminal
node tools/screenshots.cjs http://localhost:8001
```

Les captures sont régénérées dans [docs/maquettes/img/](#).

4

Documentation technique

Documentation technique — Vite & Gourmand

1. Réflexion technologique initiale

1.1 Choix du back-end : pourquoi PHP 8.2 sans framework ?

Le sujet ne contraint aucune technologie hormis l'usage d'une BDD relationnelle et d'une BDD NoSQL. Plusieurs options ont été évaluées :

Option	Avantages	Inconvénients
Laravel/Symfony	Productivité élevée, écosystème mature	Le jury veut vérifier que je maîtrise les fondamentaux ; framework trop "magique" pour un TP
Node.js + Express	Stack moderne	Hors de mon parcours Studi
PHP 8.2 + PDO + MVC maison ✓	Maîtrise totale du code, démontre la compréhension	Plus de code à écrire

Décision : PHP 8.2 + PDO + architecture MVC implémentée à la main, avec autoloader PSR-4 via Composer. Cela correspond exactement à la stack pédagogique de Studi.

1.2 Choix du front : pourquoi pas de framework JS ?

Le projet ne nécessite ni SPA ni interactions complexes côté client. **HTML / CSS / JS vanilla** suffit largement. Avantages :

- Aucune dépendance npm à gérer
- Page rendue côté serveur → SEO et accessibilité naturels
- Performances excellentes
- Démontre la maîtrise des fondamentaux (exigence du référentiel TP)

Pour les besoins JS spécifiques :

- **Filtres dynamiques sur la liste des menus** : `fetch()` + injection HTML
- **Calcul prix temps réel** sur le formulaire de commande : événements `input` + maths côté client
- **Graphique admin** : Chart.js via CDN (léger, ~80 ko)

1.3 Choix BDD relationnelle : MariaDB / MySQL

Imposé par l'énoncé. MariaDB 10.4 est livré avec XAMPP, donc utilisé en local. Il est interchangeable avec MySQL 8.x (même dialecte SQL). En production, peut être remplacé par PostgreSQL sans modification majeure (PDO abstrait le pilote).

1.4 Choix BDD NoSQL : MongoDB

Imposé par l'énoncé : « Les données [du graphique admin] doivent venir d'une base de données non relationnelle ».

Choisi **MongoDB** pour :

- Documentation abondante
- Driver PHP officiel
- Adapté aux **données semi-structurées** (statistiques avec historique embarqué)

Dans notre cas, MongoDB stocke :

- `commandes_par_menu` : un document par menu avec un compteur incrémenté + un tableau d'historique (date, montant)
- `logs` : audit trail (création commande, changement de statut, création employé, etc.)

C'est un usage **complémentaire** au SQL, pas concurrent.

2. Configuration de l'environnement

Pré-requis

Outil	Version	Rôle
PHP	8.2+	Exécution back-end
Extensions PHP	<code>pdo_mysql</code> , <code>mongodb</code> , <code>mbstring</code> , <code>openssl</code>	Indispensables
Composer	2.x	Gestion dépendances
MariaDB / MySQL	10.4+ / 8.0+	BDD relationnelle
MongoDB	6.x+	BDD NoSQL
Git	2.x+	Versioning

Installation locale

```
# 1. Cloner le dépôt
git clone https://github.com/EvanOROFINO/vitegourmand.git
cd vitegourmand

# 2. Installer les dépendances PHP
composer install

# 3. Configurer .env
cp .env.example .env
# (puis éditer si besoin)

# 4. Créer la BDD MySQL
mysql -u root < sql/01_schema.sql
mysql -u root < sql/02_seed.sql

# 5. Démarrer MongoDB (port par défaut 27017)
mongod --dbpath /chemin/vers/dossier/data

# 6. Lancer le serveur PHP
php -S localhost:8000 -t public
```

Stack VS Code recommandée

- Extension **PHP Intelephense**
- Extension **PHP DocBlocker**
- Extension **MySQL** (Weijan Chen)

3. Modèle conceptuel de données

Le MCD a été construit à partir de l'**Annexe 1 du sujet**, avec quelques ajouts pour respecter complètement les besoins métier.

Tables principales

```
utilisateur (utilisateur_id, email, password, prenom, nom, telephone,
            adresse_postale, ville, pays, actif, reset_token, reset_expires)

role (role_id, libelle)

possede → relation N,N entre utilisateur et role

theme (theme_id, libelle)
regime (regime_id, libelle)

menu (menu_id, titre, description, nombre_personne_minimum, prix_par_personne,
      conditions_menu, quantite_restante, theme_id, regime_id, actif)

menu_image → galerie d'images d'un menu

plat (plat_id, titre, photo, type{entree|plat|dessert})

menu_plat → relation N,N entre menu et plat

allergene (allergene_id, libelle)
plat_allergene → relation N,N entre plat et allergene

commande (numero_commande PK varchar, utilisateur_id, menu_id,
          date_commande, date_prestation, heure_livraison,
          adresse_livraison, ville_livraison, distance_km,
          nombre_personne, prix_menu, prix_livraison, reduction, prix_total,
          statut, pret_materiel, restitution_materiel,
          motif_annulation, mode_contact)

commande_statut_historique (id, numero_commande, statut, commentaire, date_changement)

avis (avis_id, utilisateur_id, numero_commande, note, description, statut)

horaire (horaire_id, jour, heure_ouverture, heure_fermeture, ordre)

contact_message (id, titre, description, email, created_at)
```

Justification des écarts avec le MCD de l'annexe

- **Numéro de commande** = `VARCHAR(50)` au lieu d'un INT auto-incrémenté → permet un format `CMD-YYYYMMDD-XXXXXX` lisible par le client
- **Table `commande_statut_historique`** ajoutée pour stocker la timeline de la commande (exigence du sujet sur la traçabilité)
- **Table `menu_image`** ajoutée pour gérer la galerie (exigence : « une galerie d'image »)

Schéma MongoDB (NoSQL)

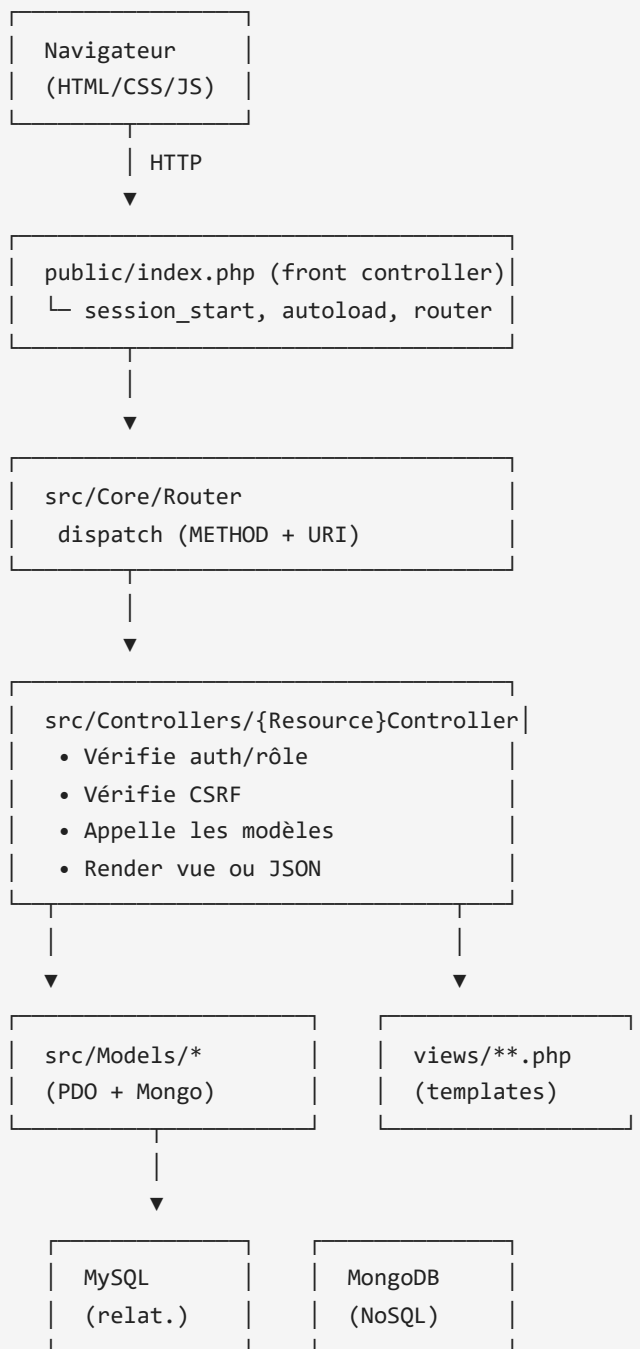
Collection `commandes_par_menu` :

```
{
  "_id": "ObjectId(...)",
  "menu_id": 1,
  "titre": "Menu Tradition Bordelaise",
  "nb_commandes": 5,
  "ca_total": 1850.00,
  "historique": [
    { "date": ISODate("2026-05-07"), "montant": 486.00 },
    ...
  ]
}
```

Collection `logs` :

```
{
  "_id": "ObjectId(...)",
  "event": "commande_creee",
  "context": { "numero": "CMD-...", "user_id": 3 },
  "date": ISODate("..."),
  "ip": "127.0.0.1"
}
```

4. Architecture applicative



Flux de requête type — Création d'une commande

1. **Front** : utilisateur soumet le formulaire `POST /commander`
2. **Routeur** : match sur `[CommandeController::class, 'submit']`
3. **CommandeController** :
 - `Auth::requireLogin()` → redirige vers `/login` si non connecté
 - `verifyCsrf()` → 419 si jeton invalide
 - Validation des champs (date future, nombre min, ville, etc.)
 - `Commande::calculerPrix()` → applique règles métier (réduction, livraison)
 - `Commande::create()` → insert MySQL transactionnel (commande + historique)

- `Stats::incrementCommandeMenu()` → upsert MongoDB
- `Stats::log()` → audit trail MongoDB
- `Mailer::send()` → mail de confirmation
- `flash('success', ...)` + redirect

5. Sécurité

Risques OWASP top 10 et mitigations

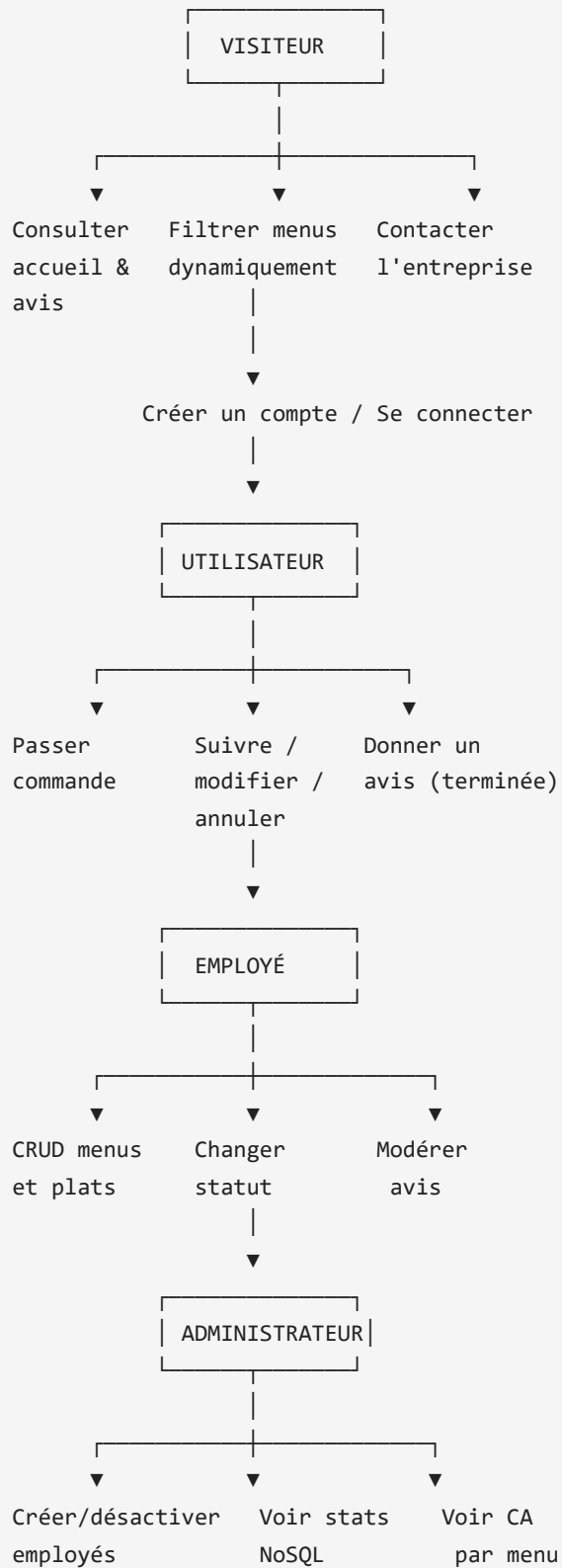
Risque	Mitigation
A01 — Broken access control	Méthodes <code>Auth::requireRole()</code> sur chaque action protégée
A02 — Cryptographic failures	<code>password_hash</code> avec bcrypt, pas de mots de passe en clair, <code>.env</code> exclu de git
A03 — Injection SQL	Requêtes préparées PDO avec paramètres nommés (zéro concat)
A03 — Injection XSS	Échappement HTML via <code>e()</code> partout, CSP en HTTP header
A04 — Insecure design	Mot de passe fort imposé (10 car., 4 classes), tokens uniques
A05 — Security misconfiguration	Erreurs masquées en prod (<code>APP_DEBUG=false</code>), <code>.htaccess</code> désactive <code>Indexes</code> , headers de sécurité
A07 — Identification & auth	<code>session_regenerate_id</code> au login, <code>samesite=Lax</code> , <code>httponly</code> , expiration tokens reset (1h)
A08 — Software integrity	<code>composer.lock</code> versionné, dépendances minimales
A10 — SSRF	Aucune fonctionnalité prenant des URL utilisateur

Secrets

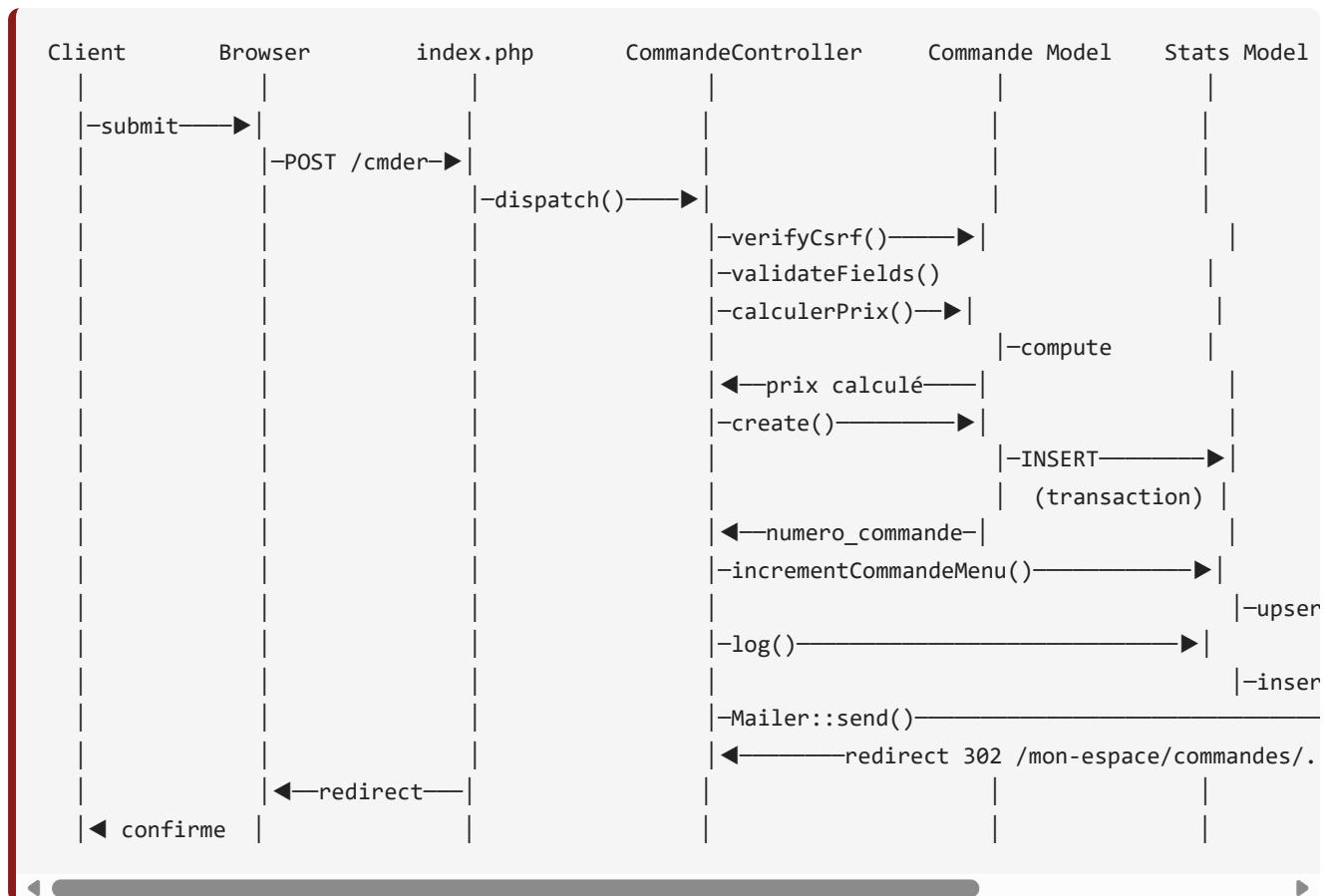
Le fichier `.env` est dans `.gitignore` . La clé `APP_SECRET` est générée aléatoirement en production.

6. Diagrammes UML

6.1 Diagramme de cas d'utilisation (Use Case)



6.2 Diagramme de séquence — Passage de commande



7. Documentation du déploiement

Plateformes cibles

Trois plateformes ont été envisagées :

Plateforme	PHP	MySQL	MongoDB	Coût
Railway	✅ via buildpack	✅ plugin	✅ via Atlas	gratuit (limité)
fly.io	✅ via Dockerfile	✅ via plugin	✅ via Atlas	gratuit (limité)
Render	✅ web service	✅ plugin	✅ via Atlas	gratuit (limité)

Choix retenu : (à compléter au moment du déploiement)

Variables d'environnement à définir en production

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://vitegourmand.example.com
APP_SECRET=<générer une clé aléatoire de 64 caractères>

DB_HOST=<host fourni par le provider>
DB_NAME=vitegourmand
DB_USER=<user>
DB_PASSWORD=<password>

MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/
MONGO_DB=vitegourmand_nosql

MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=...
MAIL_PASSWORD=...
```

Étapes de déploiement (Railway exemple)

1. Créer un nouveau projet Railway connecté au repo GitHub `vitegourmand`
2. Ajouter un plugin **MySQL** → noter les variables `MYSQLHOST`, `MYSQLDATABASE`, etc.
3. Créer un compte **MongoDB Atlas** gratuit, créer un cluster M0, noter l'URI
4. Ajouter les variables d'env dans Railway
5. Configurer le **build command** : `composer install --no-dev --optimize-autoloader`
6. Configurer le **start command** : `php -S 0.0.0.0:$PORT -t public`
7. Push sur la branche `main` → déclenche le build
8. **Initialiser la BDD MySQL** : se connecter au shell Railway, exécuter `mysql < sql/01_schema.sql &&`
`mysql < sql/02_seed.sql`
9. Vérifier l'URL publique

Domaine personnalisé (optionnel)

Possibilité d'ajouter un domaine perso via les DNS du registrar (CNAME vers Railway).

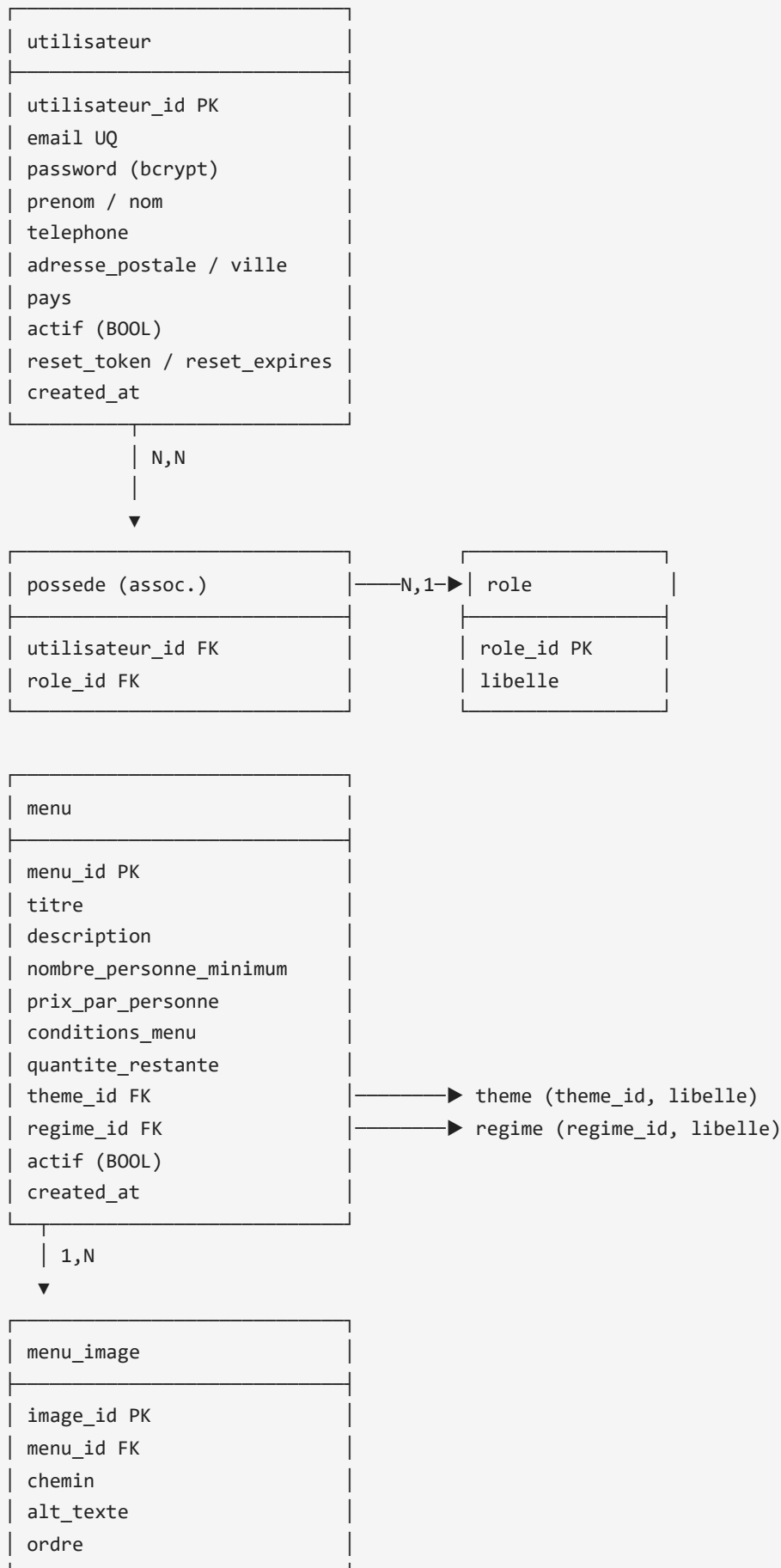
5

Modèle Conceptuel de Données

MCD — Vite & Gourmand

Modèle conceptuel des données — base relationnelle MariaDB / MySQL.

Diagramme entité-relation (textuel)



menu_plat (assoc. N,N)
menu_id FK
plat_id FK

→ menu
 → plat

plat
plat_id PK
titre
photo
type ENUM(entree plat dessert)

| N,N
 ▼

plat_allergene
plat_id FK
allergene_id FK

→ allergene (allergene_id, libelle)

commande
numero_commande PK (varchar)
utilisateur_id FK
menu_id FK
date_commande
date_prestation
heure_livraison
adresse_livraison
ville_livraison
distance_km
nombre_personne
prix_menu
prix_livraison
reduction
prix_total
statut
pret_materiel (BOOL)
restitution_materiel (BOOL)
motif_annulation
mode_contact

→ utilisateur
 → menu

| 1,N
 ▼

commande_statut_historique
id PK

numero_commande FK
statut
commentaire
date_changement

avis
avis_id PK
utilisateur_id FK
numero_commande FK UQ
note (1-5)
description
statut ENUM(en_attente valide refuse)
created_at

—► utilisateur
 —► commande (1 avis par commande max)

horaire
horaire_id PK
jour UQ
heure_ouverture
heure_fermeture
ordre

contact_message
id PK
titre
description
email
created_at

Cardinalités principales

Relation	Cardinalité
utilisateur ↔ role	(0,N) — (1,N) via <code>possede</code>
menu ↔ theme	(1,1) — (0,N)
menu ↔ regime	(1,1) — (0,N)
menu ↔ plat	(0,N) — (0,N) via <code>menu_plat</code>
plat ↔ allergene	(0,N) — (0,N) via <code>plat_allergene</code>
menu ↔ image	(0,N) — (1,1)
commande ↔ utilisateur	(1,1) — (0,N)
commande ↔ menu	(1,1) — (0,N)
commande ↔ historique	(1,N) — (1,1)
commande ↔ avis	(0,1) — (1,1)

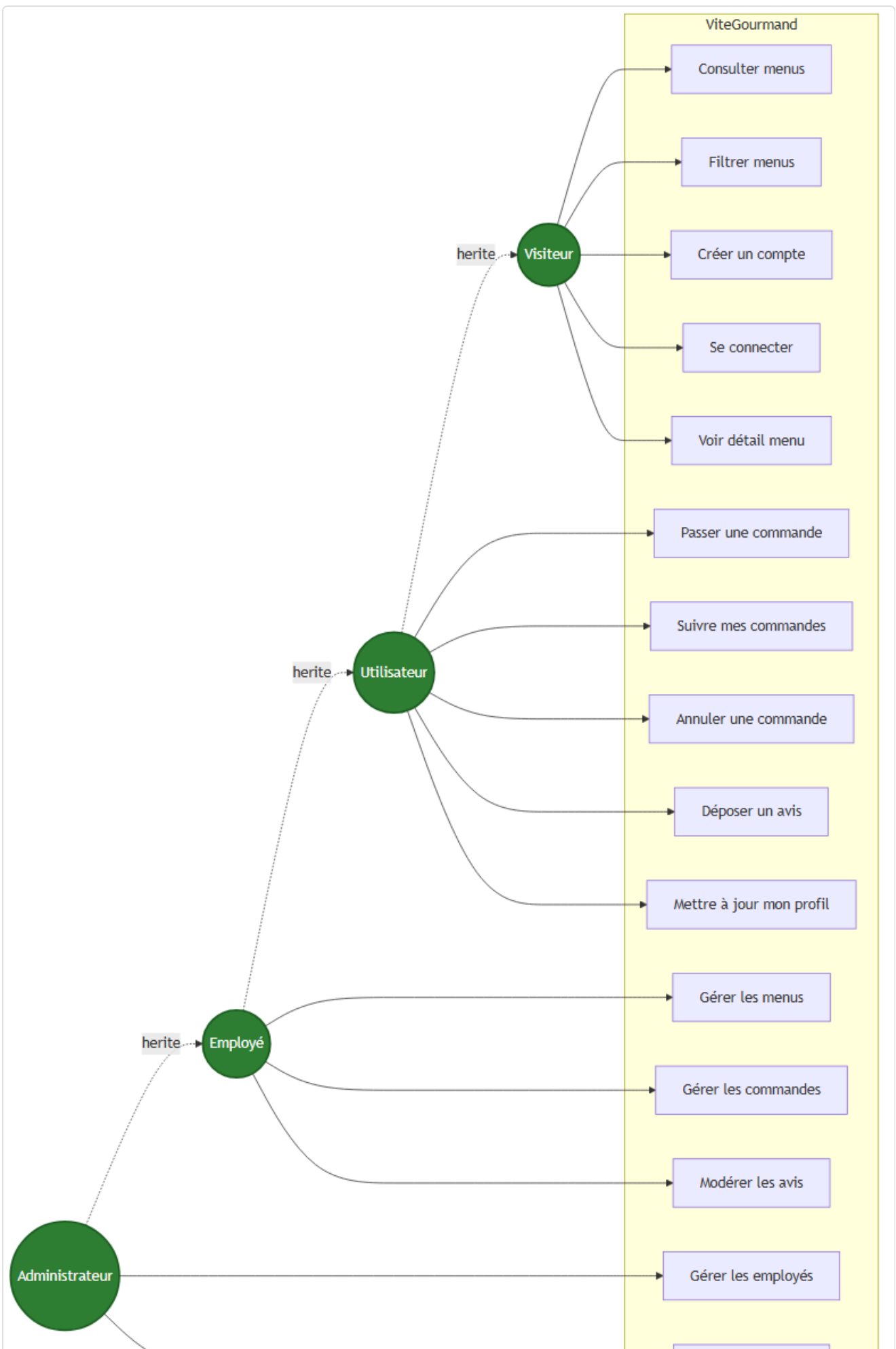
Décisions de modélisation justifiées

1. `numero_commande` en `VARCHAR(50)` plutôt qu'INT auto-incrémenté → Format lisible pour le client (`CMD-20260507-A1B2C3`), insensible aux requêtes par énumération.
2. **Table** `commande_statut_historique` **séparée** → Tracer chronologiquement chaque transition (exigence du sujet : « Le suivi de la commande énumère tous les états de sa commande suivi de la date et l'heure de modification »).
3. `utilisateur.actif` en **BOOLEAN** → Permet de désactiver un compte employé sans le supprimer (préserve les FKs et l'historique).
4. **Validation note 1-5 côté PHP, pas via CHECK CONSTRAINT** → MariaDB 10.4 a montré une instabilité avec les CHECK CONSTRAINT nommées + FK. Validation déplacée dans le contrôleur (sécurité maintenue).
5. **Tables d'association nommées** (`menu_plat` , `plat_allergene` , `possede`) → Conforme aux bonnes pratiques 3FN, évite les listes encodées en VARCHAR.
6. **MongoDB pour les statistiques** (collection `commandes_par_menu`) → Document avec compteur incrémenté + historique embarqué = pattern idéal en NoSQL pour des analytics. **Exigence explicite du sujet.**

6

Diagramme cas d'utilisation

Diagramme de cas d'utilisation — Vite & Gourmand



[Voir statistiques](#)

Source : `use-case.mmd` — généré via Mermaid CLI.

Acteurs et relations d'héritage

Acteur	Hérite de	Description
Visiteur	—	Utilisateur non authentifié
Utilisateur	Visiteur	Client ayant créé un compte
Employé	Utilisateur	Gère menus, plats, horaires, commandes, avis
Administrateur	Employé	Crée les employés, voit les statistiques

Cas d'utilisation détaillés

Visiteur

- **Consulter les menus** : liste des menus proposés par le traiteur
- **Filtrer les menus** : par thème, régime, prix
- **Créer un compte** : email, mot de passe robuste (10 car, maj/min/chiffre/spécial), adresse, téléphone
- **Se connecter** : authentification
- **Voir détail menu** : description, plats, allergènes, prix

Utilisateur (étend Visiteur)

- **Passer une commande** : choisir menu + nombre de personnes + adresse + date prestation
- **Suivre mes commandes** : voir l'historique et le statut en cours
- **Annuler une commande** : tant qu'elle est en `acceptee` ou `en_attente`
- **Déposer un avis** : après statut `terminee`, note 1-5 + commentaire
- **Mettre à jour mon profil** : adresse, téléphone, mot de passe

Employé (étend Utilisateur)

- **Gérer les menus** : CRUD complet (titre, plats, prix, thème, régime, photos)
- **Gérer les commandes** : changer statut (`en_preparation` → `livraison` → `livré` → `terminée`)
- **Modérer les avis** : valider / refuser un avis avant publication

Administrateur (étend Employé)

- **Gérer les employés** : créer / désactiver un compte employé

- **Voir statistiques** : commandes par menu, chiffre d'affaires par période (graphiques Chart.js)

7

Gestion de projet

Gestion de projet — Vite & Gourmand

1. Méthode

J'ai opté pour une **approche agile simplifiée** inspirée de Scrum / Kanban :

- **Découpage en User Stories (US)** dérivées du sujet
- **Sprints courts** (1 semaine en moyenne) pour livrer des incréments fonctionnels
- **Tableau Kanban** pour visualiser l'avancement (**Backlog** → **En cours** → **En revue** → **Fait**)
- **Tests fréquents** après chaque US pour limiter la dette technique

Les outils choisis :

- **Trello** (ou Notion) pour le Kanban et le suivi
- **Git + GitHub** pour le versioning et la traçabilité (avec branches **main** / **develop** / **feature/***)
- **VS Code** comme IDE principal

2. Découpage en User Stories

#	User Story	Estimation	Priorité
US01	En tant que visiteur , je veux voir une page d'accueil présentant l'entreprise et les avis validés, pour me donner confiance	S	Haute
US02	En tant que visiteur , je veux consulter la liste des menus avec filtres dynamiques (prix, thème, régime, nb pers.), pour trouver rapidement un menu adapté	M	Haute
US03	En tant que visiteur , je veux consulter le détail d'un menu (composition, allergènes, conditions), pour me décider en connaissance de cause	M	Haute
US04	En tant que visiteur , je veux créer un compte avec un mot de passe sécurisé, pour pouvoir commander	M	Haute
US05	En tant qu' utilisateur , je veux me connecter et récupérer mon mot de passe en cas d'oubli, pour accéder à mon espace	S	Haute
US06	En tant qu' utilisateur authentifié , je veux passer commande avec calcul automatique du prix (réduction, livraison), pour réserver une prestation	L	Haute
US07	En tant qu' utilisateur , je veux suivre, modifier ou annuler mes commandes (selon le statut), pour garder le contrôle	M	Moyenne
US08	En tant qu' utilisateur , je veux donner un avis sur une commande terminée, pour partager mon expérience	S	Moyenne
US09	En tant qu' employé , je veux gérer les commandes (changer le statut, annuler avec motif), pour suivre les prestations	L	Haute
US10	En tant qu' employé , je veux gérer les menus, plats, horaires et modérer les avis, pour tenir le site à jour	L	Moyenne
US11	En tant qu' administrateur , je veux créer / désactiver des comptes employés, pour gérer mon équipe	M	Moyenne
US12	En tant qu' administrateur , je veux visualiser les statistiques de commandes par menu (graphique) issues d'une BDD NoSQL, pour piloter l'activité	M	Moyenne
US13	En tant qu' administrateur , je veux calculer le CA par menu et par période, pour produire mes rapports	M	Moyenne
US14	En tant que visiteur , je veux contacter l'entreprise via un formulaire, pour obtenir des renseignements	S	Basse

Estimation : **S** = simple (≤ 4 h), **M** = moyen (4–8 h), **L** = long (≥ 1 jour)

3. Roadmap / Sprints

Sprint 1 — Setup & fondations (semaine 1)

- Installation environnement (PHP, Composer, MySQL, MongoDB)
- Architecture MVC, routeur, autoloader PSR-4
- Schéma SQL + seed
- Page d'accueil (US01)
- Mentions légales / CGV

Sprint 2 — Cœur fonctionnel public (semaine 2)

- Liste des menus + filtres dynamiques (US02)
- Détail menu (US03)
- Création de compte + connexion (US04, US05)

Sprint 3 — Tunnel de commande (semaine 3)

- Formulaire de commande avec calcul prix temps réel (US06)
- Espace utilisateur : suivi des commandes (US07)
- Système d'avis avec modération (US08)
- Mails automatiques (bienvenue, confirmation, terminé)

Sprint 4 — Espaces internes (semaine 4)

- Espace employé : gestion commandes et statuts (US09)
- Gestion menus / plats / horaires (US10)
- Modération avis (US10)
- Espace admin : gestion employés (US11)
- Statistiques NoSQL avec graphique (US12)
- CA par menu et période (US13)
- Formulaire de contact (US14)

Sprint 5 — Polish, tests, livrables (semaine 5)

- Tests manuels exhaustifs (visiteur / utilisateur / employé / admin)
- Corrections bugs + accessibilité RGAA
- Charte graphique : palette, typo, maquettes (3 desktop + 3 mobile)
- Documentation : manuel utilisateur, doc technique
- Déploiement et tests en ligne

4. Tableau Kanban (état final)

Fait

- US01 — Page d'accueil

- US02 — Liste menus + filtres dynamiques
- US03 — Détail menu
- US04 — Création de compte
- US05 — Connexion / mdp oublié
- US06 — Tunnel de commande + calcul prix
- US07 — Suivi / modification / annulation commande
- US08 — Avis 1-5 étoiles avec modération
- US09 — Gestion commandes côté employé
- US10 — CRUD menus, plats, horaires + modération avis
- US11 — Gestion employés admin
- US12 — Stats par menu (MongoDB + Chart.js)
- US13 — CA par menu et période
- US14 — Formulaire de contact

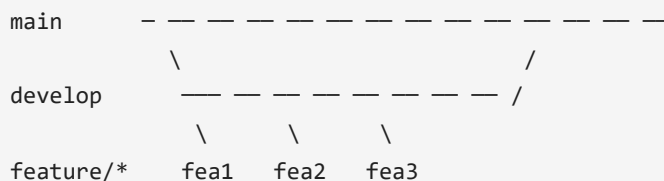
En revue

- Tests d'accessibilité RGAA approfondis
- Optimisation des images

À faire (post-livraison)

- Internationalisation (FR/EN)
- Mode panier multi-menus (1 commande = plusieurs menus)
- Paiement en ligne (Stripe)

5. Stratégie Git



Règles :

- Branche `main` = code stable et déployé
- Branche `develop` = intégration des features avant validation
- Une branche par feature (`feature/listing-menus` , `feature/tunnel-commande` , etc.)
- Merge dans `develop` après revue + tests
- Merge dans `main` quand `develop` est validé sur tous les flux

6. Risques identifiés et mitigations

Risque	Impact	Mitigation
Crash du serveur MySQL (CHECK CONSTRAINT non supporté MariaDB 10.4)	Bloquant	Validation déplacée côté PHP + type <code>TINYINT</code> simple
Perte de données utilisateur	Critique	Backups SQL via <code>mysqldump</code> planifiés
Faible XSS via avis	Sécurité	Échappement HTML systématique (<code>htmlspecialchars</code>)
Faible CSRF sur formulaires	Sécurité	Jetons CSRF unique par session vérifiés sur chaque POST
Faible SQL injection	Sécurité	Requêtes préparées PDO partout (audit code OK)
Mots de passe trop faibles	Sécurité	Politique stricte 10 car. + 4 classes
Mail jamais reçu (filtré spam)	UX	Logs en dev + SMTP authentifié en prod
Indisponibilité MongoDB	Fonctionnel (admin)	Fallback : afficher un message "stats indisponibles"

7. Bilan personnel

Ce TP m'a permis de mettre en pratique l'ensemble des compétences du référentiel DWWM :

Compétence référentiel	Mise en œuvre dans le projet
Installer / configurer son environnement	XAMPP + MongoDB + Composer + Git + VS Code
Maquetter des interfaces	Charte graphique + maquettes desktop/mobile
Réaliser des interfaces statiques	HTML5 sémantique + CSS3 (custom)
Développer la partie dynamique	JS vanilla (filtres dynamiques, calcul prix) + Chart.js
Mettre en place une BDD relationnelle	Schéma normalisé en 3FN, contraintes FK, index
Composants d'accès SQL et NoSQL	PDO + driver MongoDB
Composants métier serveur	Modèles + contrôleurs avec règles métier (calcul prix, statuts)
Documenter le déploiement	Doc technique complète + guide d'installation README

Le plus gros défi a été de **concilier la richesse fonctionnelle du sujet** (3 rôles distincts, 5 statuts de commande, règles tarifaires complexes, conformité RGAA + RGPD, exigence NoSQL) avec **la maîtrise totale du code** (sans framework). L'architecture MVC maison, héritée du projet EcoRide, m'a fait gagner un temps précieux.

8

Kanban

Tableau Kanban — Vite & Gourmand

État au **2026-05-07** (livraison du projet).

✅ **Fait (14 / 14 user stories implémentées)**

US01 · Page d'accueil

- ☒ Hero avec photo et CTA
- ☒ Présentation 3 atouts (expérience, produits locaux, sur-mesure)
- ☒ Section avis clients (uniquement validés)

US02 · Liste des menus + filtres dynamiques

- ☒ Grid responsive (3 cols desktop, 1 col mobile)
- ☒ 5 filtres : prix min, prix max, thème, régime, nb personnes
- ☒ Actualisation **sans rechargement de page** (fetch + injection HTML)

US03 · Détail d'un menu

- ☒ Galerie d'images
- ☒ Composition par catégorie (entrée / plat / dessert)
- ☒ Liste agrégée des allergènes
- ☒ Conditions du menu **mises en évidence**
- ☒ Bouton commande (ou redirection login si visiteur)

US04 · Création de compte

- ☒ Formulaire complet (nom, prénom, email, GSM, adresse, ville, pays)
- ☒ Validation mot de passe **10 car. min, 1 maj, 1 min, 1 chiffre, 1 spécial**
- ☒ Mail de bienvenue automatique
- ☒ Rôle `utilisateur` attribué automatiquement

US05 · Connexion / mot de passe oublié

- ☒ Login avec email + mot de passe
- ☒ Lien "Mot de passe oublié" → mail avec token (1h de validité)
- ☒ Page de réinitialisation

US06 · Tunnel de commande

- ☒ Pré-remplissage avec infos du compte
- ☒ Date de prestation J+1 minimum
- ☒ Calcul prix temps réel (côté client + côté serveur)
- ☒ **Réduction 10%** automatique si nb_pers ≥ minimum + 5

- ☒ **Frais livraison 5€ + 0,59€/km** hors Bordeaux
- ☒ Numéro de commande unique format `CMD-YYYYMMDD-XXXXXX`
- ☒ Mail de confirmation
- ☒ Stock décrémenté

US07 · Suivi / modification / annulation

- ☒ Liste des commandes utilisateur
- ☒ Détail avec **timeline des statuts**
- ☒ Annulation possible si statut `en_attente`
- ☒ Modification possible si statut `en_attente` (sauf le menu choisi)

US08 · Système d'avis

- ☒ Note 1-5 étoiles + commentaire
- ☒ Disponible uniquement après statut `terminee`
- ☒ **Modération** par employé avant publication
- ☒ Avis validés visibles sur la page d'accueil

US09 · Gestion commandes (employé)

- ☒ Dashboard avec liste filtrée
- ☒ Filtre par statut + recherche client/n° commande
- ☒ Changement de statut avec workflow complet
- ☒ Annulation avec **motif** + **mode de contact** obligatoires

US10 · CRUD menus / plats / horaires + modération avis (employé)

- ☒ CRUD complet sur les menus (titre, descr, prix, conditions, stock, plats inclus)
- ☒ CRUD plats (entrée / plat / dessert)
- ☒ Édition des horaires d'ouverture
- ☒ Validation / refus des avis en attente

US11 · Gestion employés (admin)

- ☒ Création employé avec mdp défini par l'admin
- ☒ Mail de notification (sans le mot de passe en clair, comme exigé par l'énoncé)
- ☒ Désactivation / réactivation
- ☒ **Pas de création admin depuis l'app** (conformité au sujet)

US12 · Statistiques NoSQL (admin)

- ☒ Collection MongoDB `commandes_par_menu` mise à jour à chaque commande
- ☒ Graphique en barres avec **Chart.js**
- ☒ Données issues exclusivement de la BDD non-relationnelle (exigence sujet)

US13 · CA par menu et par période (admin)

- ☒ Filtres date début / date fin / menu
- ☒ KPIs en haut (total commandes, total CA)
- ☒ Tableau détaillé par menu

US14 · Formulaire de contact

- ☒ Formulaire titre + email + message
- ☒ Stockage en BDD pour traçabilité
- ☒ Mail à l'équipe Vite & Gourmand

☒ Conformité

- ☒ RGPD : mentions légales, droit d'accès / rectification / suppression mentionné
- ☒ RGAA : skip-link, aria-labels, contrastes WCAG AA, navigation clavier
- ☒ Sécurité : OWASP top 10 audité (CSRF, XSS, SQLi, sessions, etc.)
- ☒ Mots de passe : politique stricte (10 car., 4 classes)
- ☒ Comptes admin : non créables depuis l'app

☒ Livrables

- ☒ Code sur GitHub PUBLIC
- ☒ README avec instructions complètes
- ☒ Fichiers SQL (`01_schema.sql` + `02_seed.sql`)
- ☒ Charte graphique PDF (palette, typo, maquettes)
- ☒ Manuel d'utilisation PDF
- ☒ Documentation technique PDF (MCD, diagrammes, déploiement)
- ☒ Documentation gestion de projet PDF (méthode, kanban)
- ☒ Application déployée en ligne (*à compléter avec l'URL*)
- ☒ Branches Git `main` + `develop` + `feature/*`

À faire (post-livraison, hors scope ECF)

- ☐ Tests automatisés (PHPUnit)
- ☐ Internationalisation FR / EN
- ☐ Panier multi-menus
- ☐ Paiement en ligne (Stripe)
- ☐ Mode hors-ligne (PWA)
- ☐ Accès employé à un calendrier des prestations